

Islamic University of Technology

Department of Computer Science and Engineering

Software Project Lab II (SPL-2)

MatrixCanvas

An Interactive Linear Algebra Visualizer

Supervisor

Anika Tabassum

Team Members

Faiyaz Tahmid 230042107

Abrar Faiyaz 230042108

Ittesaf Ithun 230042125

Tamim Naser 230042133

Group Number

Group 15

July 27, 2026

Contents

1	Project Overview	2
2	Motivation	3
3	User Requirement Analysis	4
3.1	Stakeholders	4
3.1.1	Stakeholder 1 — Undergraduate Students (Primary)	4
3.1.2	Stakeholder 2 — Course Instructors	4
3.1.3	Stakeholder 3 — Independent Self-Directed Learners (Global)	4
3.2	Functional Requirements	5
3.3	Non-Functional Requirements	5
4	System Diagrams	6
4.1	User Interaction Flowchart	6
4.2	Class Diagram	7
5	Key Features	8
5.1	Feature 1 — Real-Time Matrix Transform Playground	8
5.2	Feature 2 — Transformable Shape Overlay	8
5.3	Feature 3 — Step-by-Step Decomposition Animations	8
5.4	Feature 4 — Structured Learning Pathway	9
5.5	Feature 5 — 3D Visualization Mode	9
5.6	Feature 6 — Properties Panel & Shareable Permalinks	9
6	Tools and Technology	10
7	Proposed Timeline	11
8	Action Taken Based on Feedback	12

Project Overview

MatrixCanvas is a browser-based interactive visualization tool designed to make linear algebra intuitive, explorable, and genuinely engaging. Inspired by tools like Desmos, it allows users to input any matrix and immediately observe how it transforms two-dimensional — and optionally three-dimensional — space. The coordinate grid deforms in real time, geometric shapes warp according to the transformation, and smooth animations illustrate the transition from the identity matrix to any entered matrix.

Beyond the free-form exploration playground, MatrixCanvas includes a structured learning pathway system. A complete beginner can start from foundational concepts such as vectors and scalar multiplication, and progressively work through scaling, rotation, shear, determinants, eigenvectors, and major matrix decompositions: Singular Value Decomposition (SVD), LU Decomposition, QR Decomposition, and Eigendecomposition. Each topic is paired with an interactive visualization that renders the concept geometric and tangible rather than purely symbolic.

The application is entirely frontend-based — no backend infrastructure, server, or user account is required. It will be deployed as a publicly accessible web application. A three-dimensional mode powered by Three.js extends the tool to 3×3 matrix transformations, allowing users to observe how matrices act on objects in $\mathbb{R}^3$.

Motivation

Linear algebra is one of the most foundational subjects in undergraduate engineering and computer science education. Yet it remains one of the most persistently misunderstood. Most students encounter it through rows of abstract symbols, mechanical row-reduction, and definitions that provide little geometric intuition. A student can correctly compute an eigenvalue without having any real sense of what it *means* for a matrix to stretch space along an invariant axis.

The gap is not one of intelligence — it is one of *representation*. Concepts such as the determinant (area and volume scaling), eigenvectors (directions that remain invariant under transformation), and Singular Value Decomposition (a composition of rotate-scale-rotate operations) are fundamentally geometric. Yet textbooks and lectures rarely make this visible in an interactive way.

The 3Blue1Brown “Essence of Linear Algebra” YouTube series demonstrated that when students *see* transformations animated, comprehension and retention improve dramatically. However, video content is passive. MatrixCanvas makes the same conceptual insight interactive — students can manipulate matrices themselves, place their own shapes, and observe consequences in real time. This active, exploratory engagement transforms rote computation into genuine understanding.

As second-year students who have completed an introductory linear algebra course, our motivation is grounded in direct personal experience. We repeatedly encountered situations where a visual, interactive tool would have accelerated understanding of a concept that took days to internalize through symbolic manipulation alone. MatrixCanvas is the tool we wished had existed.

User Requirement Analysis

Stakeholders

Stakeholder 1 — Undergraduate Students (Primary)

Second-semester undergraduate students enrolled in introductory linear algebra courses form the core user base. These users typically have a basic calculus background but limited geometric intuition for matrix operations.

Requirements:

- A visual, interactive interface to explore matrix transformations without writing any code
- Immediate feedback when matrix values are changed
- Structured lessons followable independently outside class hours
- A clean, uncluttered UI that requires no prior tutorial

Stakeholder 2 — Course Instructors

Instructors can use MatrixCanvas as a live in-class demonstration tool — projecting transformations while teaching rotation matrices, null spaces, or SVD.

Requirements:

- Quickly enter a specific matrix and animate it for a classroom audience
- Pre-loaded preset matrices for common pedagogical examples (45° rotation, projection, shear)
- Shareable URLs encoding a specific canvas state for distribution to students
- Mathematically accurate visualizations suitable for academic use

Stakeholder 3 — Independent Self-Directed Learners (Global)

Anyone worldwide studying linear algebra outside a formal course — through MOOCs, textbooks, or self-study.

Requirements:

- Free, open access with no account or login required
- A learning pathway starting from zero prerequisites
- Coverage of advanced topics such as SVD and QR decomposition
- Compatibility with any modern browser, including tablet and mobile devices

Functional Requirements

1. Users shall be able to input a 2×2 or 3×3 matrix via an editable UI panel
2. The coordinate grid shall deform in real time to reflect the entered matrix transformation
3. Users shall be able to place geometric shapes (rectangle, triangle, custom polygon) and observe their transformation
4. The system shall animate the transformation from the identity matrix to the entered matrix
5. The system shall display computed matrix properties: determinant, rank, trace, and invertibility flags
6. Users shall be able to follow a sequential learning pathway from beginner to decomposition-level topics
7. Each lesson shall programmatically control the canvas state (pre-loading matrix, shape, and animation)
8. Users shall be able to visualize Eigendecomposition, SVD, LU, and QR as step-by-step animated sequences
9. Users shall be able to toggle between 2D and 3D visualization modes
10. A shareable permalink shall encode the current canvas state in the URL

Non-Functional Requirements

1. The application shall load in under 3 seconds on a standard broadband connection
2. All animations shall maintain a minimum of 30 frames per second
3. The UI shall be fully responsive on tablet and desktop screen sizes
4. The application shall require no installation, backend server, or user account
5. All mathematical computations shall be numerically stable and accurate to at least 6 decimal places

System Diagrams

User Interaction Flowchart

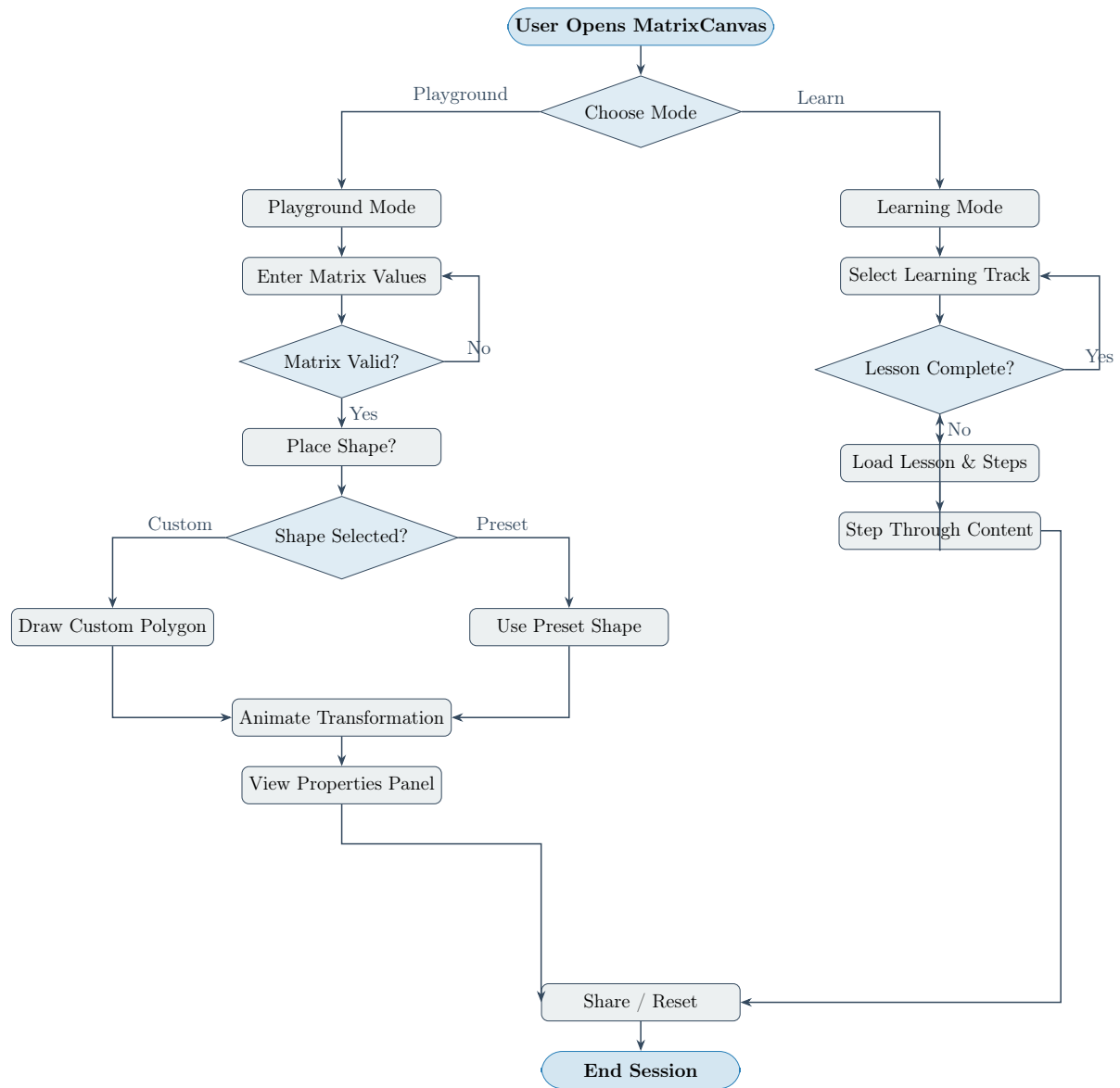


Figure 4.1: User Interaction Flowchart

Class Diagram

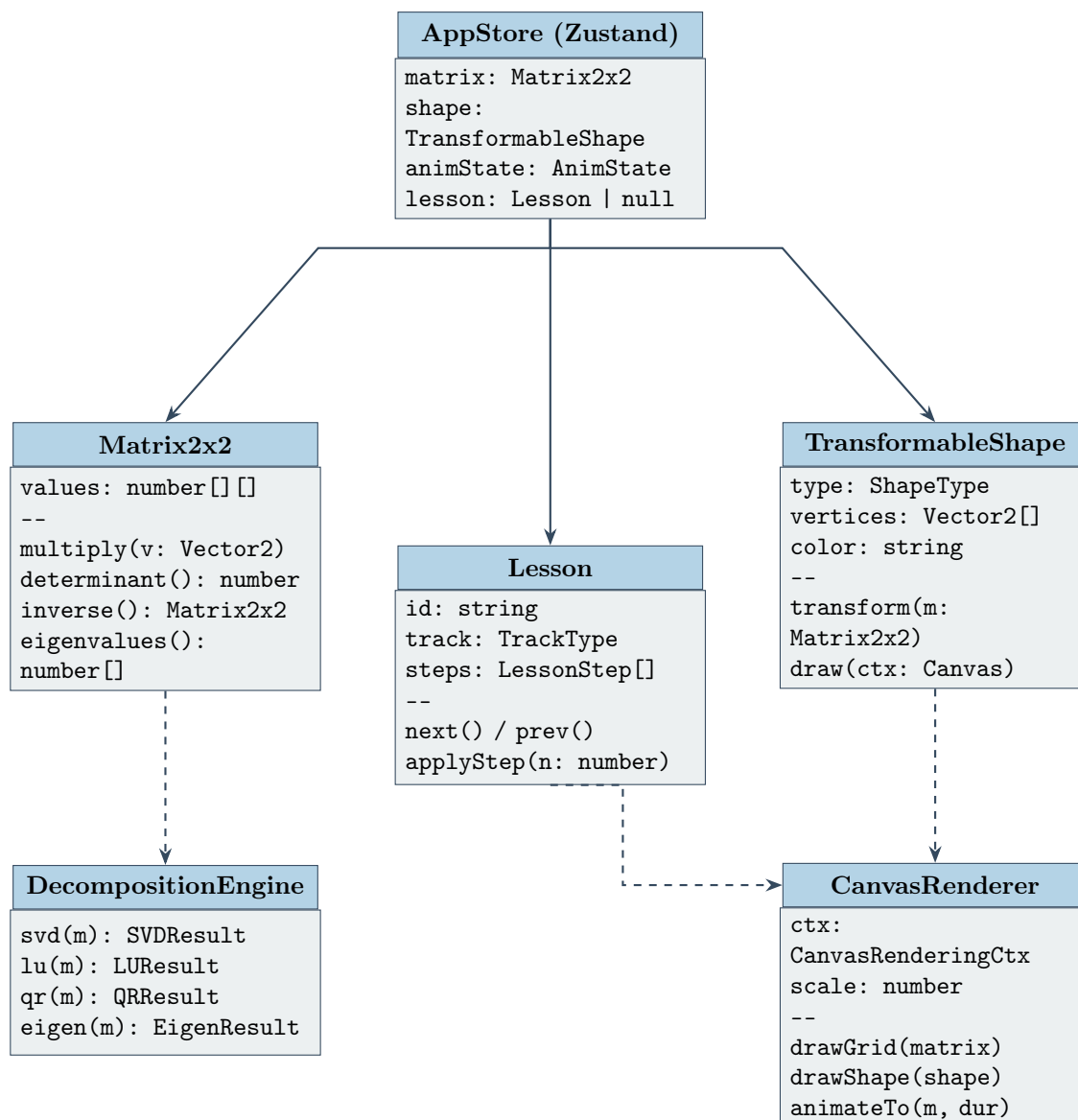


Figure 4.2: Class Diagram — Core System Architecture

Key Features

Feature 1 — Real-Time Matrix Transform Playground

The core feature of MatrixCanvas. A user enters values into a 2×2 matrix input panel and the coordinate grid deforms immediately to reflect the resulting linear transformation. The transformed positions of every grid-line intersection are computed via matrix-vector multiplication and re-rendered at every keystroke.

Use Case: A student wants to understand what a rotation matrix does geometrically. They enter the values for a 45° rotation, press “Animate,” and watch the entire grid rotate smoothly around the origin. The basis vectors $\hat{i}$ and $\hat{j}$ are highlighted as they move to their new positions — the columns of the entered matrix.

Feature 2 — Transformable Shape Overlay

Users can place a geometric shape (rectangle, triangle, or custom polygon) onto the coordinate grid before applying a transformation. The shape is rendered in its original position as a faded ghost outline, and in its transformed position as a solid colored overlay. The area of both shapes is displayed numerically, creating a direct visual link to the determinant.

Use Case: An instructor places a unit square on the grid and enters a shear matrix with determinant equal to 1. The class observes that although the square becomes a parallelogram, its area is preserved — making the geometric meaning of the determinant immediately evident without any algebra.

Feature 3 — Step-by-Step Decomposition Animations

For each major decomposition, the matrix is broken into its component factors and each is animated as a distinct transformation step on the canvas.

- **SVD:** $A = U\Sigma V^T$ — animated as: rotate (V^T), scale along axes (Σ), rotate again (U)
- **Eigendecomposition:** $A = PDP^{-1}$ — change of basis, scale along eigenvectors, change back
- **LU:** Gaussian elimination visualized as a sequence of applied shear transformations
- **QR:** Gram-Schmidt orthogonalization animated as column-by-column orthonormalization

Use Case: A student who has memorized the SVD formula but does not understand it watches the three-step animation and realizes that every matrix, no matter how complex, is simply a composition of rotations and axis-aligned scaling operations.

Feature 4 — Structured Learning Pathway

A curated, sequential lesson system organized into three progressive tracks. Each lesson programmatically loads a matrix, shape, and animation sequence into the canvas, paired with written explanations and typeset mathematics rendered via KaTeX.

- **Beginner:** Vectors, Matrix Multiplication, Identity, Scaling, Rotation, Shear, Reflection
- **Intermediate:** Matrix Composition, Determinant, Null Space, Column Space, Invertibility, Eigenvectors
- **Decompositions:** Eigendecomposition, SVD, LU, QR / Gram-Schmidt

Use Case: A self-learner with no prior linear algebra experience starts at Lesson 1 of the Beginner track and completes the full pathway over two weeks, using MatrixCanvas as their sole resource.

Feature 5 — 3D Visualization Mode

A Three.js powered scene allowing users to input 3×3 matrices and observe their effect on three-dimensional objects — vectors, cubes, and coordinate planes — with orbit camera controls for free navigation around the scene.

Use Case: A student studying 3D rotation matrices for a computer graphics course inputs a rotation matrix for 30° around the Y-axis and watches a cube rotate in real time, building spatial intuition that a textbook formula alone cannot provide.

Feature 6 — Properties Panel & Shareable Permalinks

A live panel displays computed matrix properties that update on every keystroke: determinant, rank, trace, and Boolean flags for invertibility, orthogonality, and symmetry — each with a plain-English description. The current canvas state is encoded into the URL as query parameters, enabling one-click sharing of any specific transformation.

Tools and Technology

Category	Technology	Purpose
Framework	React 18 + Type-Script	Component-driven UI with type-safe math logic
Build Tool	Vite	Fast development server and production bundler
2D Rendering	HTML5 Canvas 2D API	Real-time grid deformation and shape rendering
3D Rendering	Three.js	3×3 matrix transformations on 3D objects
Math Engine	mathjs	Matrix operations, SVD, LU, QR, eigenvalue decomposition
Animation	GSAP	Smooth, timeline-based transform interpolation
Math Type-setting	KaTeX	Rendering mathematical equations inside lesson content
State Management	Zustand	Shared application state across all components
Routing	React Router v6	Navigation between Playground and Learning pages
Styling	Tailwind CSS	Responsive and consistent UI development
Version Control	Git + GitHub	Collaboration, history, and source control
Deployment	Vercel	Zero-config public hosting with CI/CD

Table 6.1: Tools and Technologies

Proposed Timeline

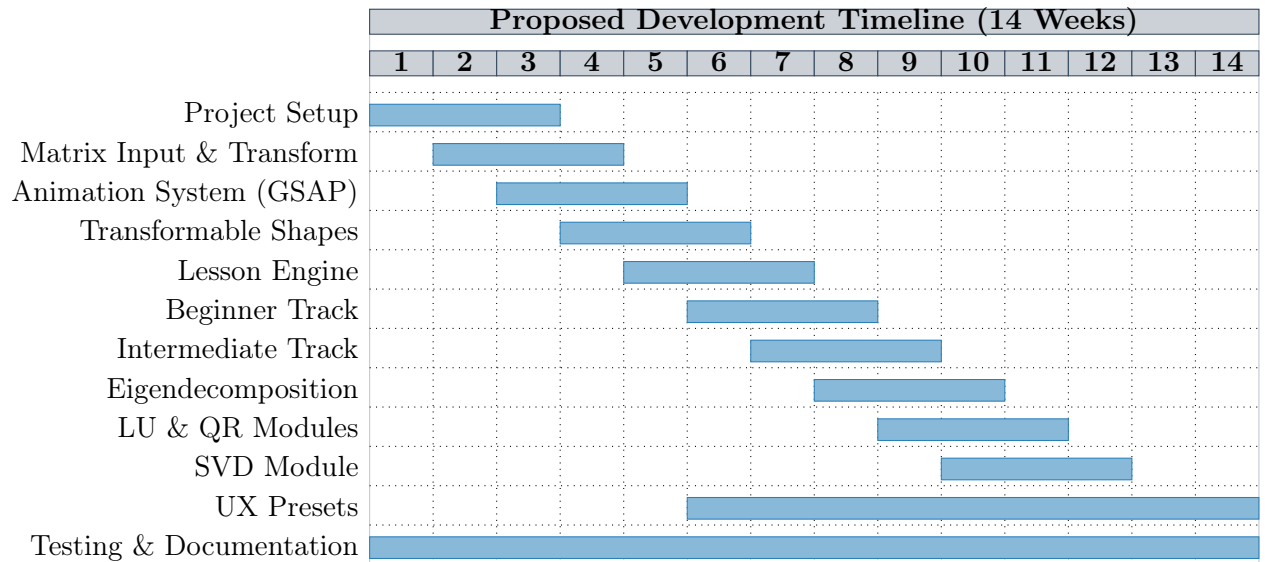


Figure 7.1: Gantt Chart — 14-Week Development Timeline

Action Taken Based on Feedback

Following the initial proposal review, feedback was received that a tool limited exclusively to two-dimensional matrix transformations may be insufficient in scope for a fourteen-week project, and would not adequately challenge the team or serve the needs of more advanced users.

Action Taken:

In response, the project scope has been formally extended to include a full **3D visualization mode** powered by Three.js, allocated to Week 12 of the development timeline. This addition addresses the feedback on the following grounds:

1. **Engineering scope:** The 3D mode introduces a meaningfully distinct rendering pipeline (Three.js vs Canvas 2D), a 3×3 matrix input system, orbit camera controls, and support for 3D transformable objects (cube, vector in $\mathbb{R}^3$). This collectively represents a substantial increase in implementation complexity.
2. **Educational value:** Many applied domains that students encounter — computer graphics, robotics, physics simulation — operate in three dimensions. A 3D mode extends the tool's relevance beyond a purely theoretical classroom context and into practical engineering applications.
3. **Stakeholder coverage:** The addition directly serves students in computer graphics and 3D geometry courses, expanding the tool's audience beyond introductory linear algebra students alone.

The 3D mode is deliberately kept outside the scope of the timeline as we want to properly accomplish the 2D portion first. After progress update, we might reconsider adding it back depending on how much progress we are able to make.